

Software Design



De La Salle University
D A S M A R I Ñ A S

Engr. Michael C. Olivo

Introduction to Software Design

Objectives

- ▶ To explain what design is and how various types of design deal with different aspects of a product
- ▶ To present design as problem solving and outline the roles of abstraction and modeling in design
- ▶ To place design in the software life cycle
- ▶ To survey software engineering design methods

What is Software Design?

- ▶ Software design is the process of defining **how** a system will be structured and implemented **before** writing code. It's about making intentional choices rather than letting architecture emerge by accident.
- ▶ **Key distinction:**
- ▶ **Architecture:** High-level structure, major components, their interactions
- ▶ **Design:** Detailed structure within components, class relationships, algorithms
- ▶ **Implementation:** Actual code that realizes the design

Foundational Principles

- ▶ **SOLID Principles (Object–Oriented Design)**
- ▶ **Single Responsibility:** A class should have only one reason to change
- ▶ **Open–Closed:** Open for extension, closed for modification
- ▶ **Liskov Substitution:** Subtypes must be substitutable for their base types
- ▶ **Interface Segregation:** Many client–specific interfaces are better than one general–purpose interface
- ▶ **Dependency Inversion:** Depend on abstractions, not concretions

DRY (Don't Repeat Yourself)

- ▶ Each piece of knowledge should have a single, unambiguous representation

But be careful: Don't over-abstract too early

KISS (Keep It Simple, Stupid)

- ▶ Simplicity should be a key goal in design
- ▶ Complexity increases maintenance costs exponentially

YAGNI (You Ain't Gonna Need It)

- ▶ Don't add functionality until it's necessary
- ▶ Premature generalization is worse than missing an abstraction
- ▶

Design Process & Techniques

- ▶ **Design Process Steps**
- ▶ Requirements → High-Level Design → Detailed Design → Implementation → Refinement

2. Key Design Activities

Decomposition

- ▶ Break system into manageable modules/components

Cohesion: How closely related elements within a module are

Coupling: How much modules depend on each other (aim for low coupling)

Abstraction

- ▶ Hide complex details behind simple interfaces

Example: Database abstraction layer hides SQL differences

- ▶ **Design Patterns (Common Solutions to Recurring Problems)**
- ▶ **Creational Patterns:**
- ▶ **Factory:** Create objects without specifying exact class
- ▶ **Singleton:** Ensure only one instance exists
- ▶ **Builder:** Construct complex objects step by step
- ▶ **Structural Patterns:**
- ▶ **Adapter:** Make incompatible interfaces work together
- ▶ **Decorator:** Add functionality dynamically
- ▶ **Facade:** Provide simplified interface to complex subsystem

- ▶ **Behavioral Patterns:**
- ▶ **Strategy:** Define family of algorithms, make them interchangeable
- ▶ **Observer:** One-to-many dependency between objects
- ▶ **Command:** Encapsulate request as object

3. Modeling Techniques

- ▶ **UML Basics (Visual Communication)**
- ▶ Class diagrams (structure)
- ▶ Sequence diagrams (interactions)
- ▶ State diagrams (behavior)
- ▶ **Remember:** UML is a communication tool, not documentation
- ▶

Importance of Software Design

- ▶ We live in a designed world.
- ▶ Design is economically important and effects our quality of life.
- ▶ Software is becoming ubiquitous.
- ▶ The quality of software design has important consequences that software designers should be aware of and take seriously.

Software Products

A **software product** is an entity comprised of one or more programs, data, and supporting materials and services that satisfies client needs and desires either as an independent artifact or as essential ingredient in some other artifact.

Software Design Defined

Software designers do what designers in other disciplines do, except they do it for software products.

Software design is the activity of specifying the mature and composition of software products that satisfy client needs and desire, subject to constraints.

Design as Problem Solving

- ▶ An especially fruitful way to think about design is as problem solving.
- ▶ Advantages
 - Suggests partitioning information between problem and solution
 - Emphasizes that there may be more than one good solution (design)
 - Suggests techniques such as changing the problem, trial and error, brainstorming, etc.

Abstraction

Abstraction is an important problem-solving technique, especially in software design.

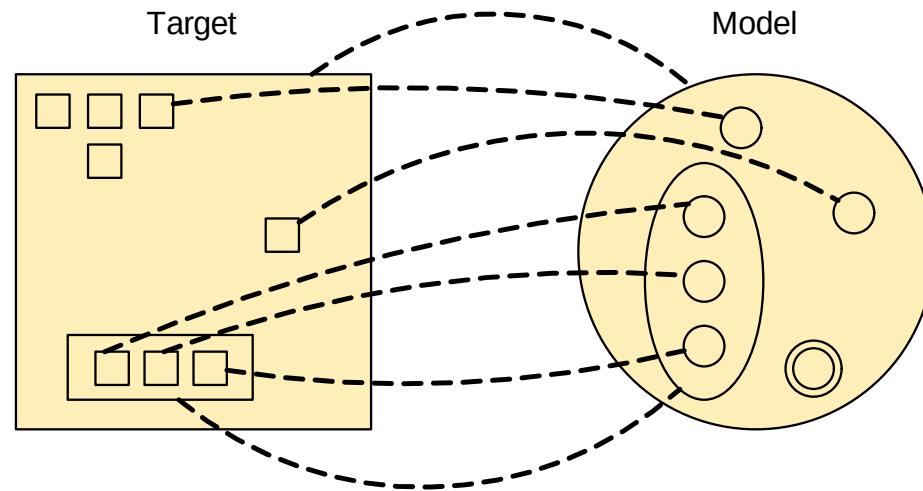
Abstraction is suppressing or ignoring some properties of objects, events, or situations in favor of others.

Importance of Abstraction

- ▶ Problem simplification
 - Abstracting allows us to focus on the most important aspects of a problem in (partially) solving it.
- ▶ Structuring problem solving
 - Top-down strategy: Solve an abstract version of the problem, then add details (refinement)
 - Bottom-up strategy: Solve parts of a problem and connect them for a complete solution

Modeling

A model represents a target by having model parts corresponding to target parts, with relationships between model parts corresponding to relationships between target parts.



Modeling in Design

- ▶ Modeling is used for the following purposes:
 - Problem understanding
 - Design creation
 - Design investigation
 - Documentation
- ▶ Modeling work because models abstract details of the target.
- ▶ Models can fail if important and relevant details are left out.

Static and Dynamic Models

A **static model** represents aspects of programs that do not change during program execution.

A **dynamic model** represents what happens during program execution.

- ▶ Static model examples include class and object models.
- ▶ Dynamic model examples include state diagrams and sequence diagrams.

Product vs. Engineering Design

- ▶ **Product designers** are concerned with styling and aesthetics, function and usability, manufacturability and manageability.
 - Industrial designers, (building) architects, interior designers, graphic designers, etc.
- ▶ **Engineering designers** are concerned with technical mechanisms and workings.
 - Structural, mechanical, chemical, and electrical engineers
- ▶ Design teams often include both product and engineering designers.

Software Product Design

Software product design is the activity of specifying software product features, capabilities, and interfaces to satisfy client needs and desires.

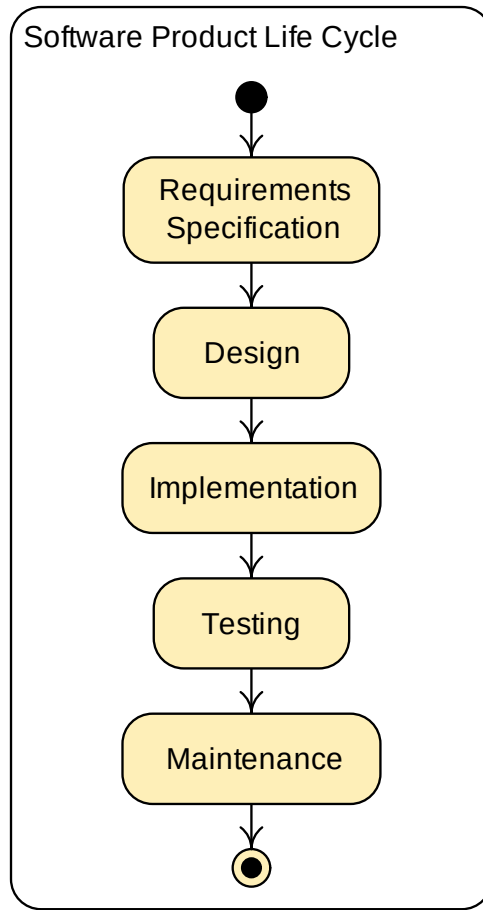
Requires skills in user interface and interaction design, communications, industrial design, and marketing

Software Engineering Design

Software engineering design is the activity of specifying programs and sub-systems, and their constituent parts and workings, to meet software product specifications.

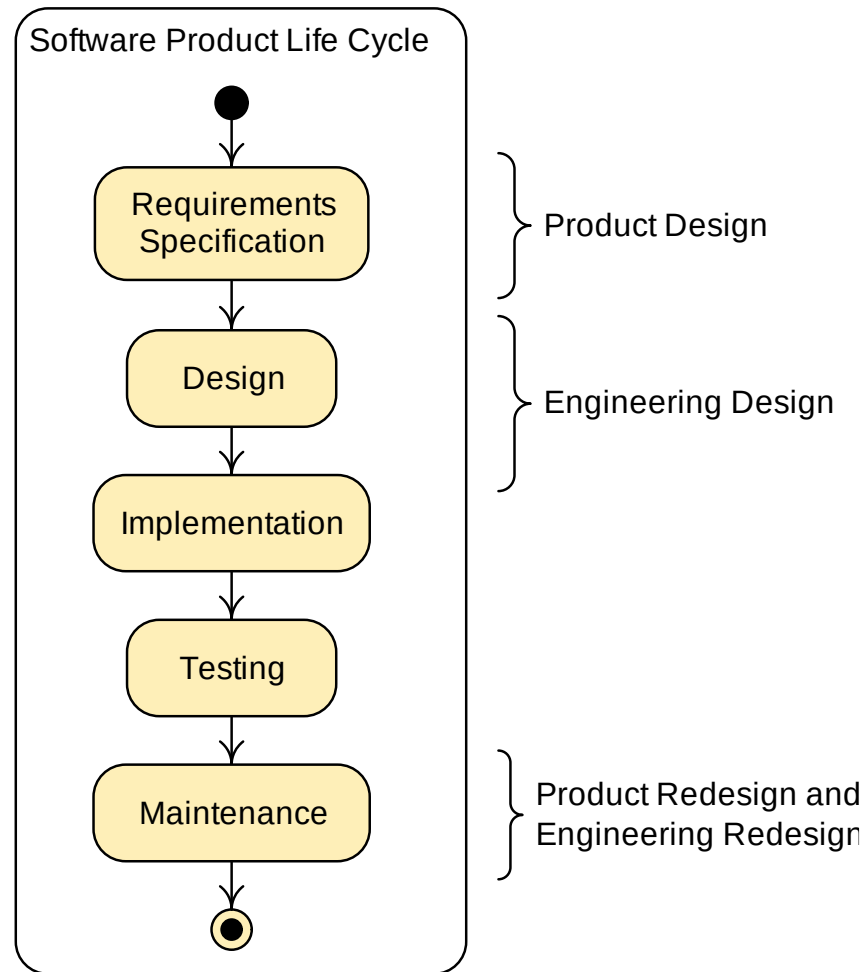
Requires skills in programming, algorithms, data structures, software design principles, practices, processes, techniques, architectures, and patterns

Waterfall Life Cycle Model

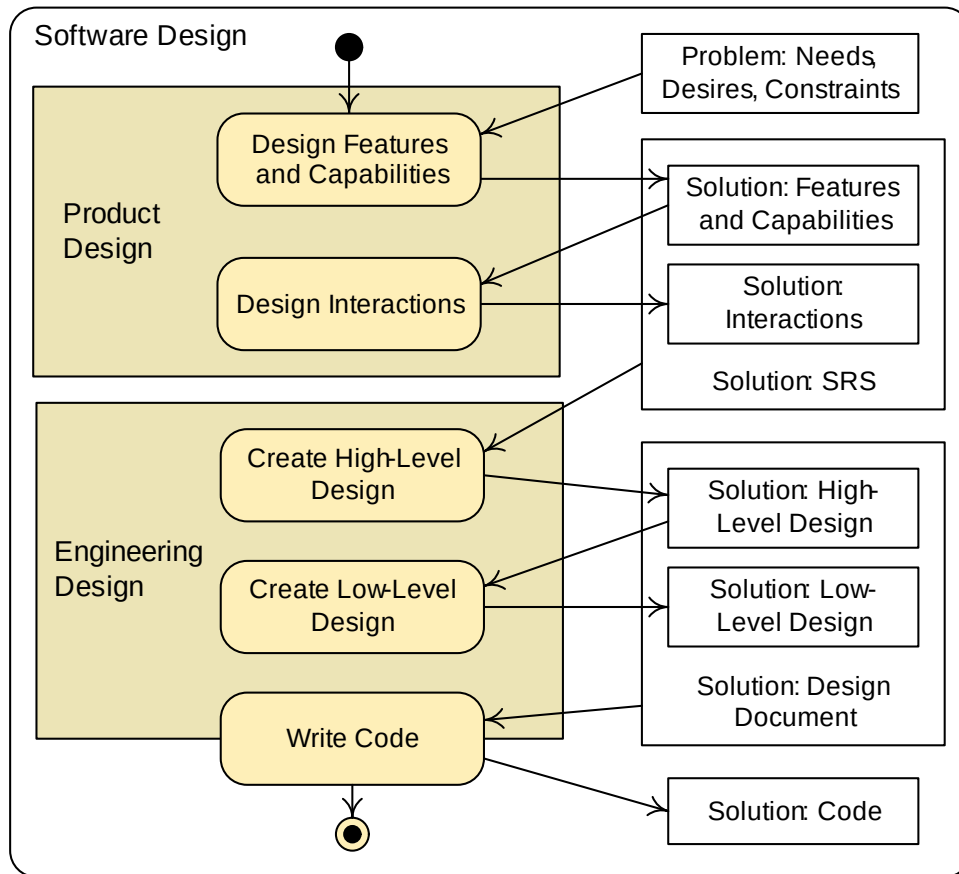


The waterfall model captures the logical, but not the temporal, relationships between software development activities.

Design Across the Life Cycle



Design Problems and Solutions



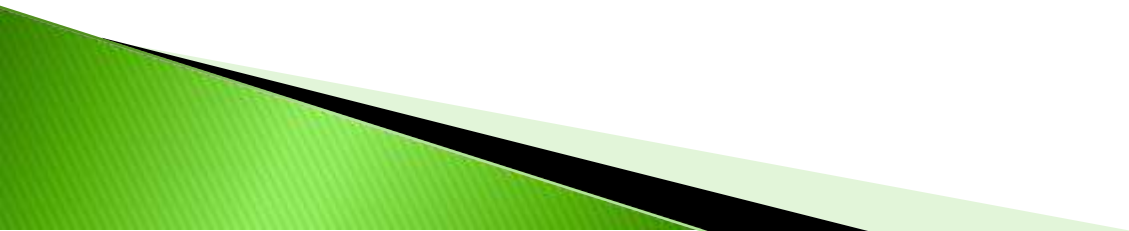
Software Design Method

A **software design method** is an orderly procedure for generating a precise and complete software design solution that meets clients needs and constraints.

Design Method Components

- ▶ *Design Process*—A collection of related tasks that transforms a set of inputs into a set of outputs
- ▶ *Design Notations*—A symbolic representational system
- ▶ *Design Heuristics*—Rules providing guidance, but no guarantee, for achieving some end
- ▶ Design methods also use **design principles** stating characteristics of design that make them better or worse.

END



Software Design



De La Salle University
D A S M A R I Ñ A S

Engr. Michael C. Olivo

Project (Class Diagram)



Unified Modeling Language, Version 2.0

- ▶ 2 types of diagrams
 - Structure Diagrams
 - Provide a way for representing the data and static relationships that are in an information system
 - Class diagram
 - Behavior Diagrams

Structure Diagrams

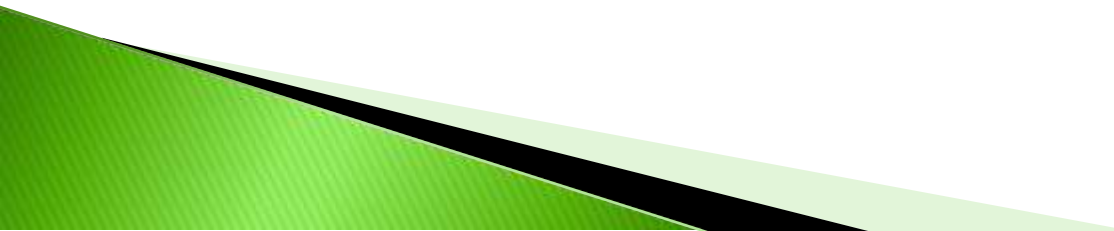
▶ Class Diagram

- Describe the structure of the system in terms of classes and objects
- Primary purpose during analysis workflow: to create a vocabulary that is used by both the analyst and users

What is a Class?

- ▶ A general template that we use to create specific instances or objects in the application domain
 - ▶ Represents a kind of person, place, or thing about which the system will need to capture and store information
 - ▶ Abstractions that specify the attributes and behaviors of a set of objects
- 

What is an Object?

- ▶ Entities that encapsulate state and behavior
 - ▶ Each object has an identity
 - It can be referred individually
 - It is distinguishable from other objects
- 

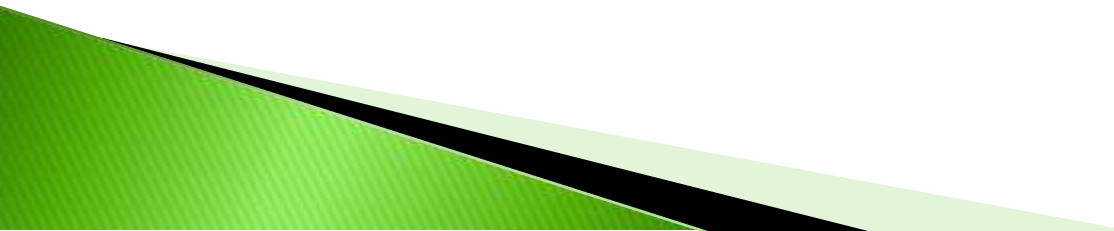
Types of Classes

- ▶ Ones found during analysis:
 - people, places, events, and things about which the system will capture information
 - ones found in application domain
- ▶ Ones found during design
 - specific objects like windows and forms that are used to build the system

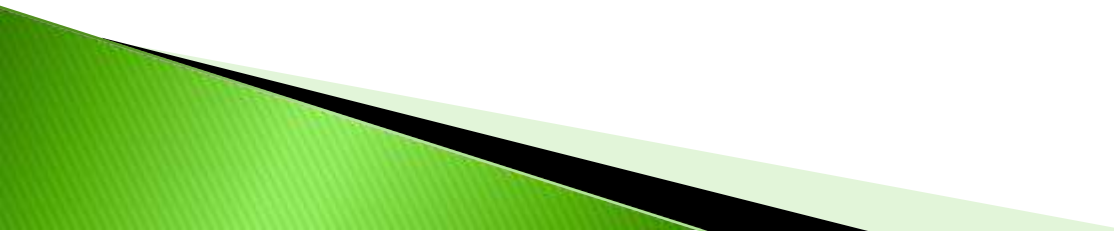
2 Kinds of Classes during Analysis

- ▶ Concrete
 - Class from application domain
 - Example: Customer class and Employee class
- ▶ Abstract
 - Useful abstractions
 - Example: Person class

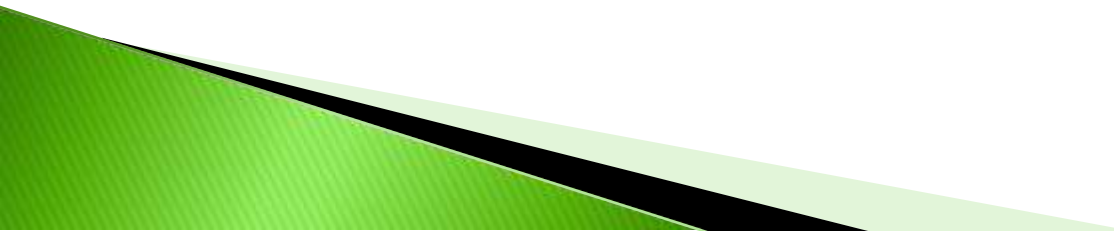
Attributes in a Class

- ▶ Properties of the class about which we want to capture information
 - ▶ Represents a piece of information that is relevant to the description of the class within the application domain
- 

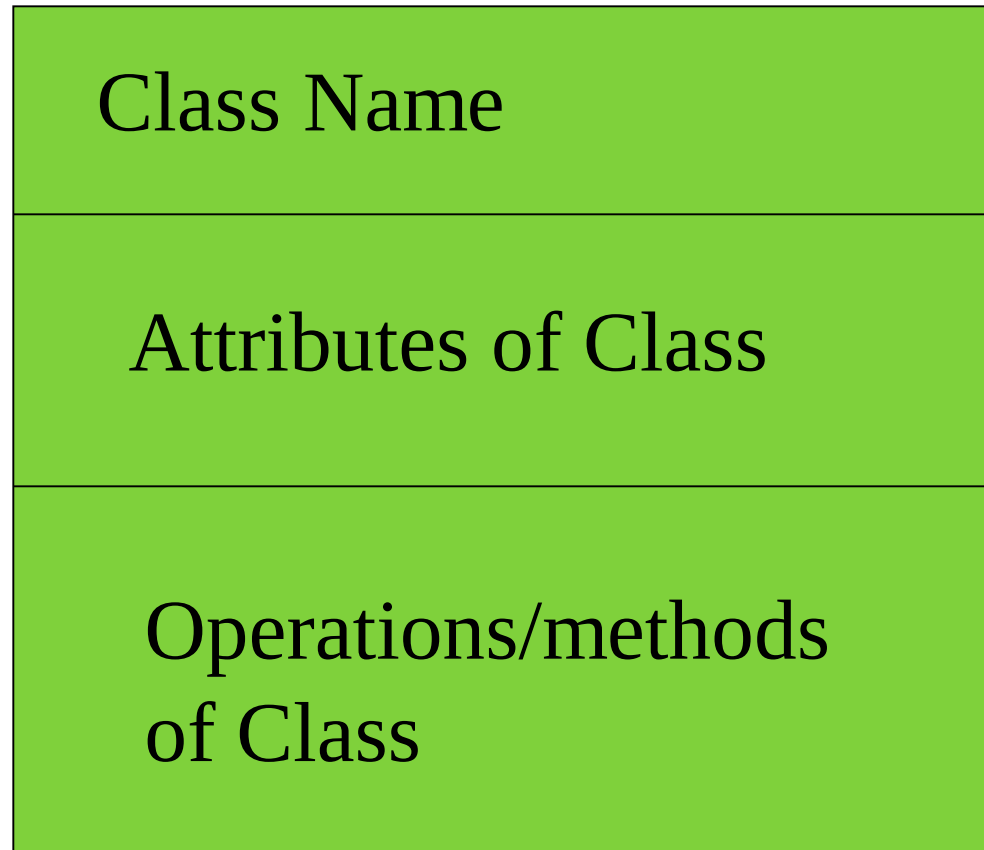
Attributes in a Class

- ▶ Only add attributes that are primitive or atomic types
 - ▶ Derived attribute
 - attributes that are calculated or derived from other attributes
 - denoted by placing slash (/) before name
- 

Operations in a Class

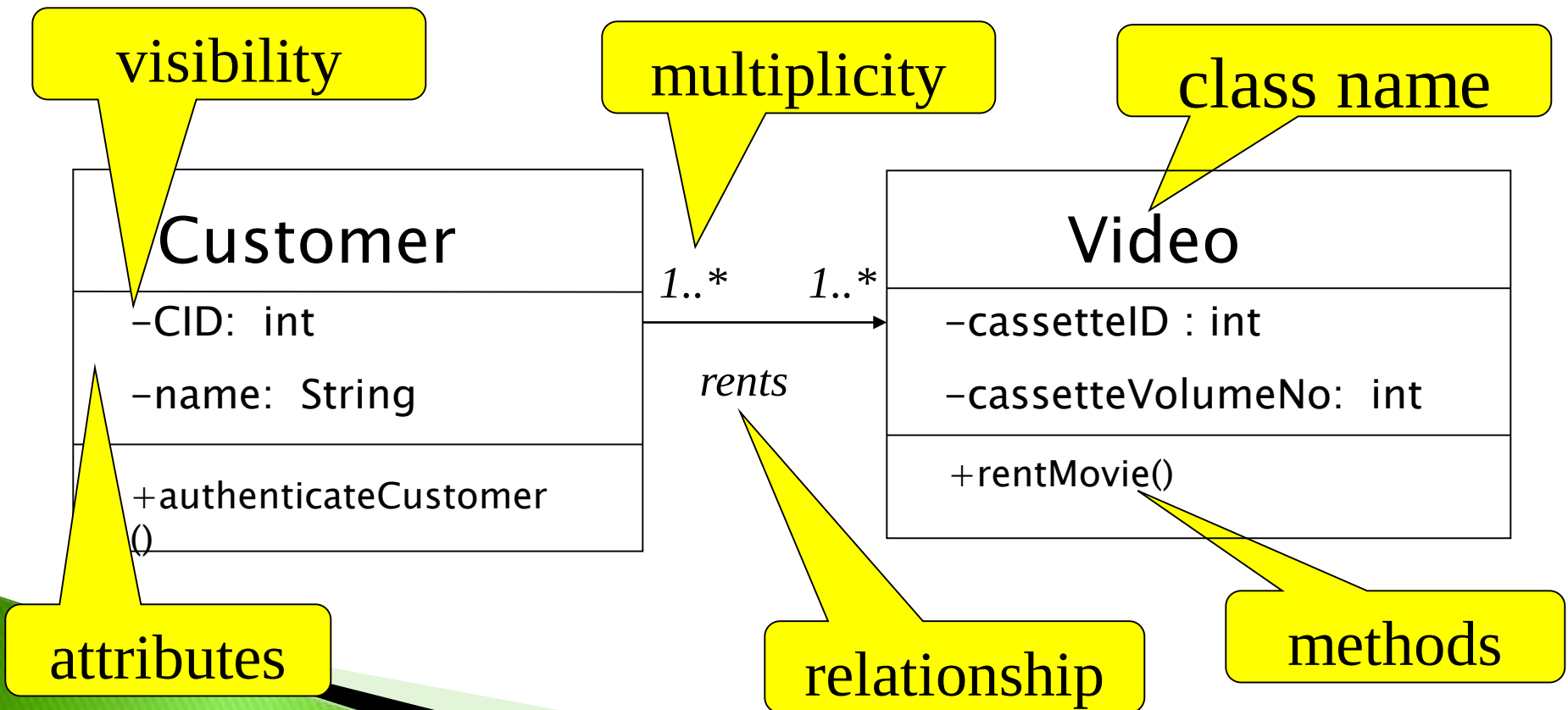
- ▶ Represents the actions or functions that a class can perform
 - ▶ Describes the actions to which the instances of the class will be capable of responding
 - ▶ Can be classified as a constructor, query, or update operation
- 

UML Representation of Class



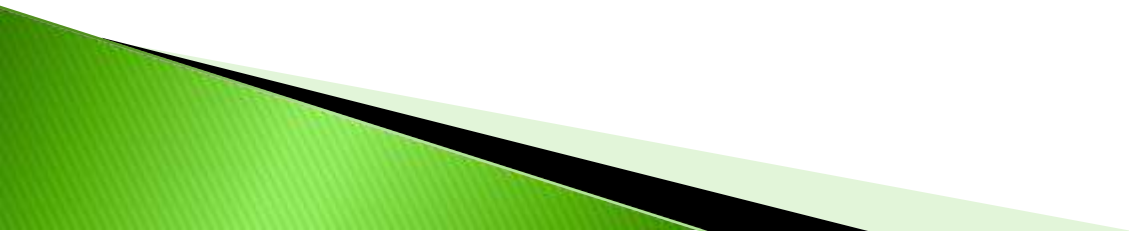
Example of a Class Diagram

Video Rental System



Visibility of Attributes and Operations

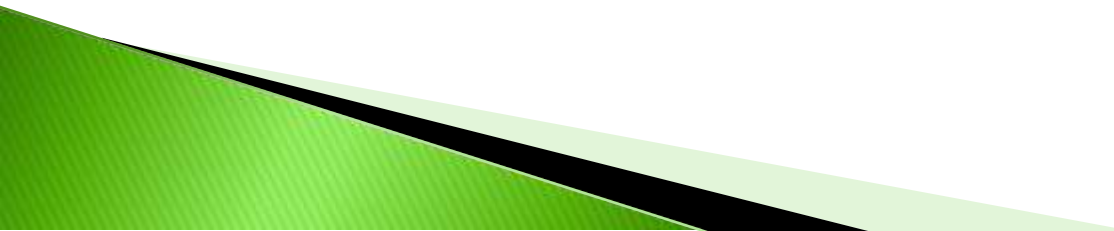
- ▶ Relates to the level of information hiding to be enforced



Visibility of Attributes and Operations

Visibility	Symbol	Accessible To
Public	+	All objects within your system.
Protected	#	Instances of the implementing class and its subclasses.
Private	-	Instances of the implementing class.

Relationships among Classes

- ▶ Represents a connection between multiple classes or a class and itself
 - ▶ 3 basic categories:
 - association relationships
 - generalization relationships
 - aggregation relationships
- 

Association Relationship

- ▶ A bidirectional semantic connection between classes
- ▶ Type:
 - name of relationship
 - role that classes play in the relationship

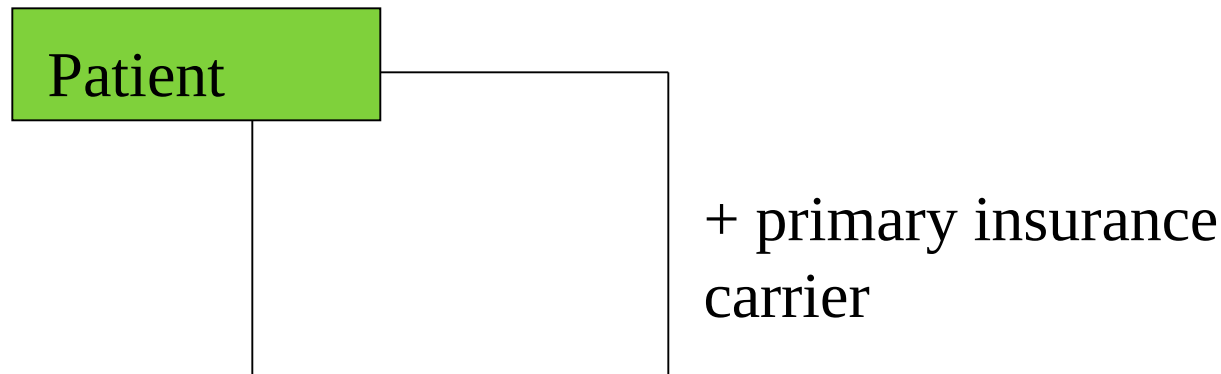
Association Relationship

- ▶ Name of relationship type shown by:
 - drawing line between classes
 - labeling with the name of the relationship
 - indicating with a small solid triangle beside the name of the relationship the direction of the association



Association Relationship

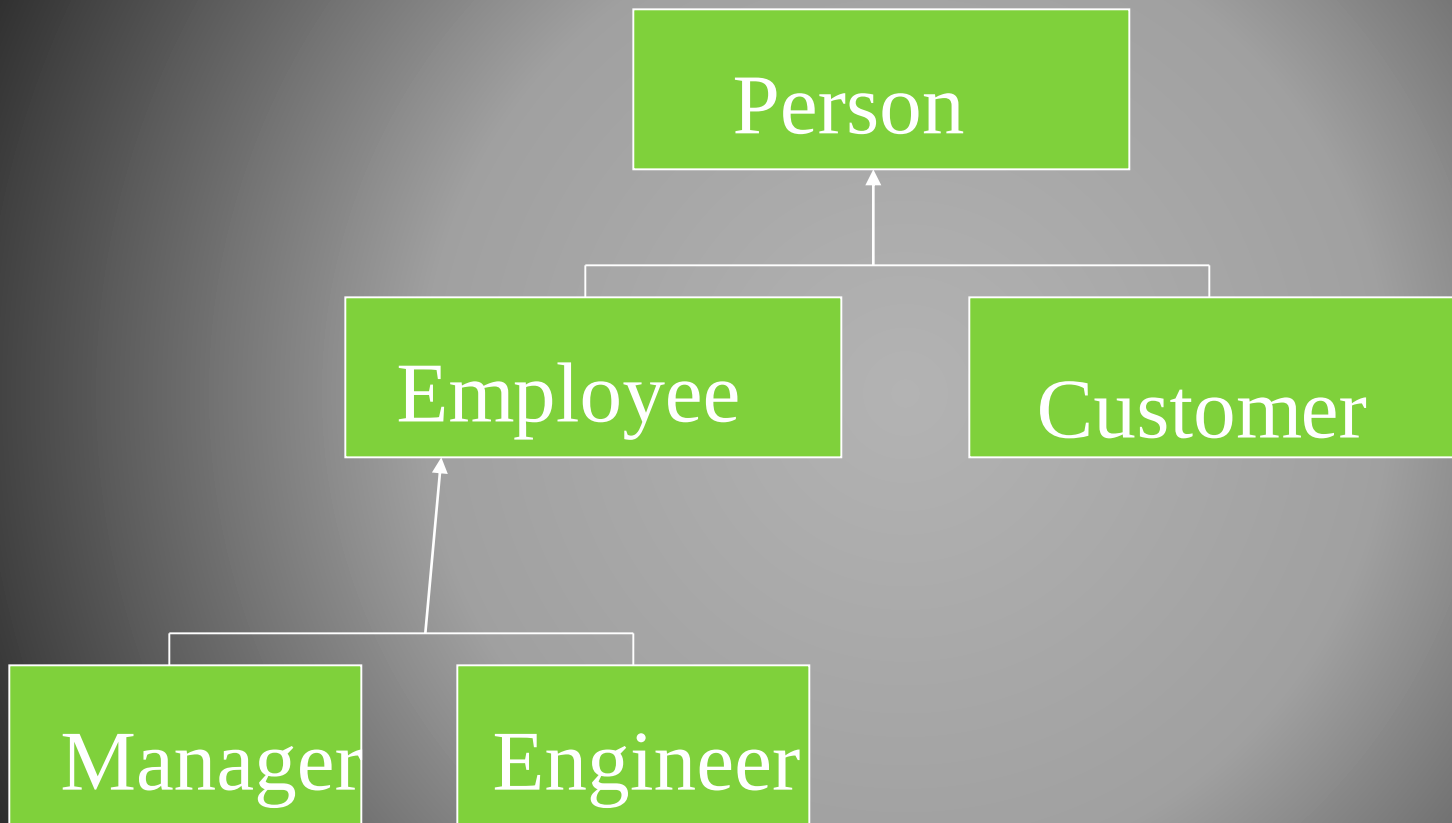
- ▶ Role type shown by:
 - drawing line between classes
 - indicating with a plus sign before the role name



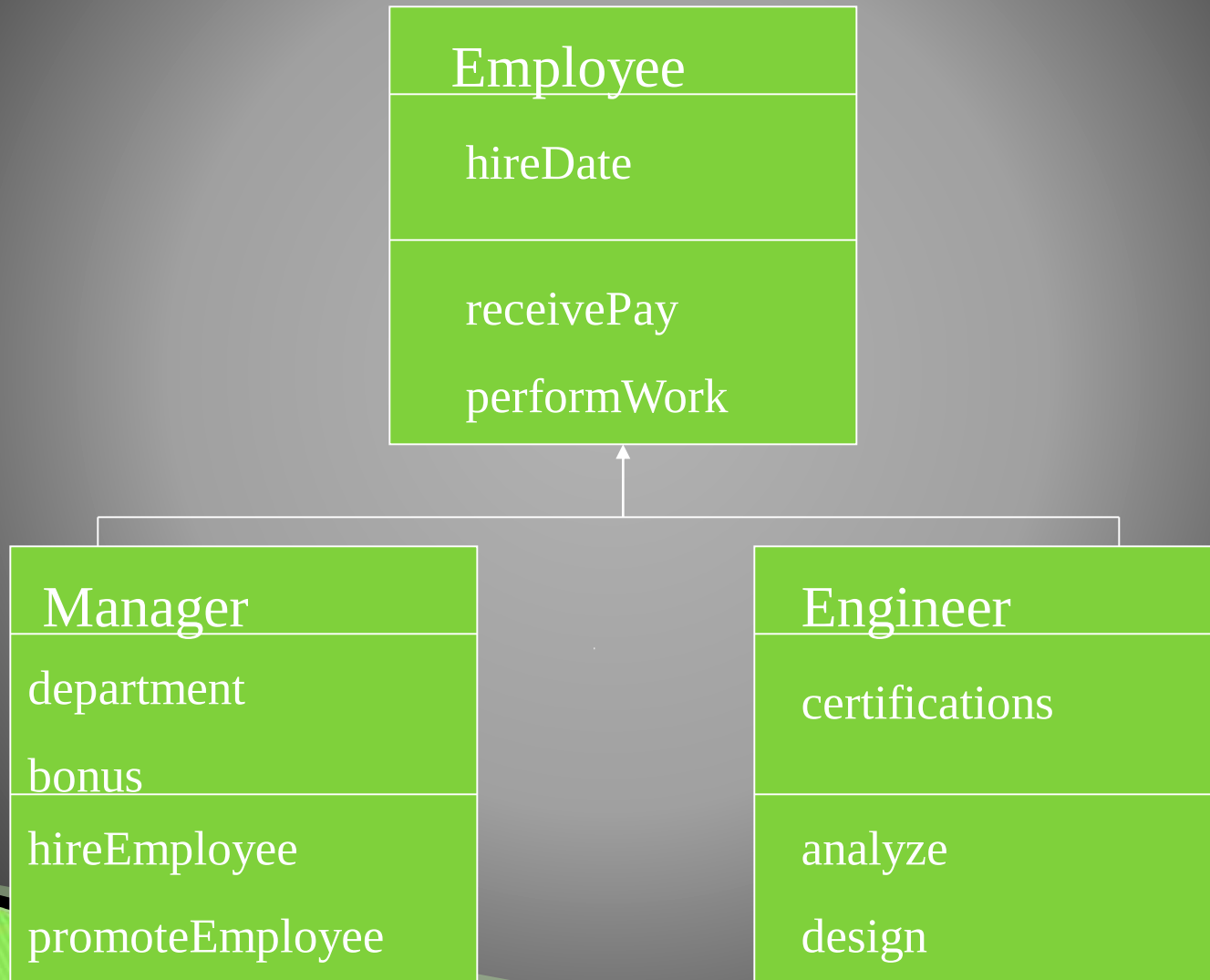
Generalization Relationship

- ▶ Enables the analyst to create classes that inherit attributes and operations of other classes
- ▶ Represented by *a-kind-of* relationship

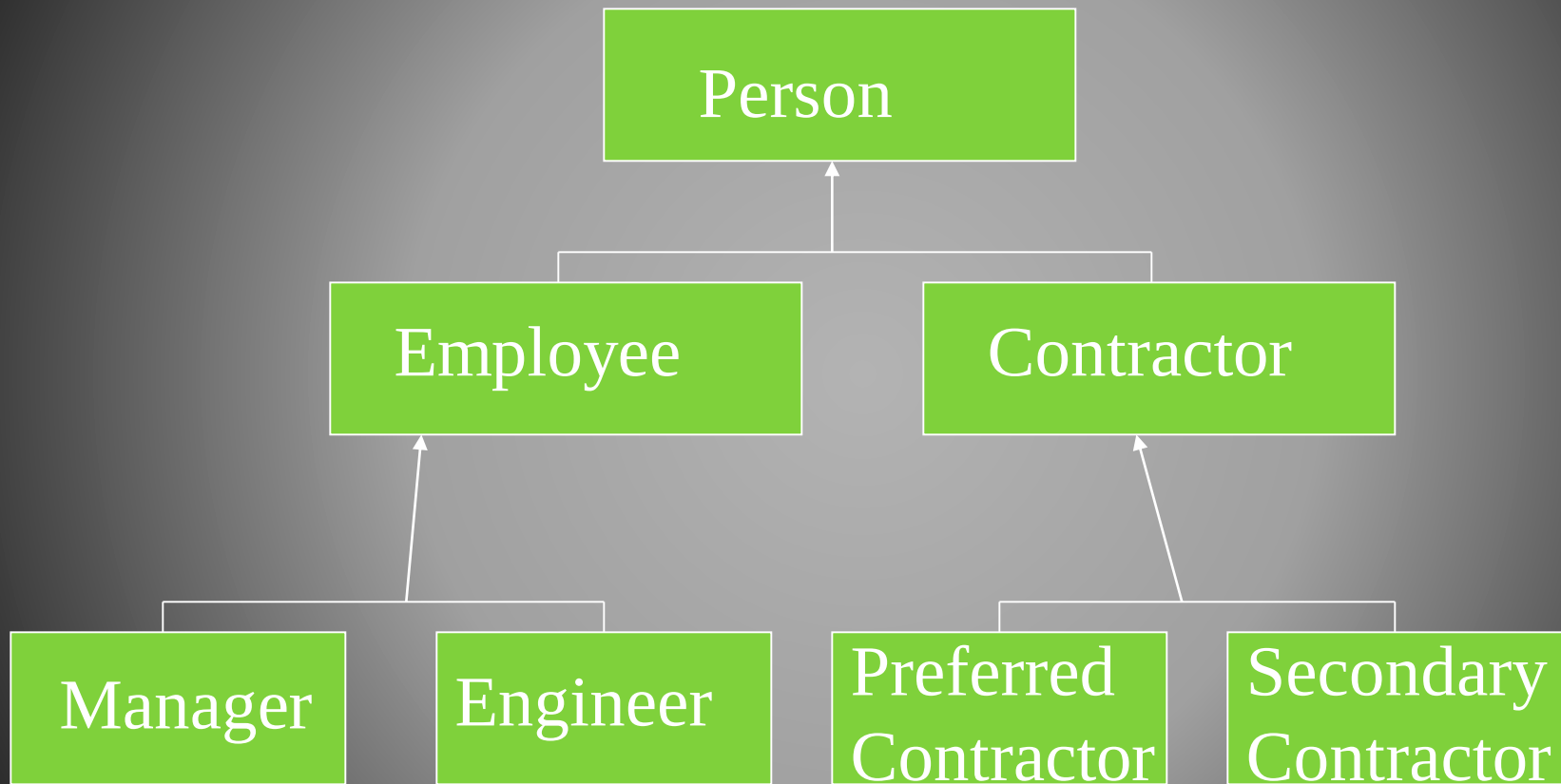
Generalization Relationship



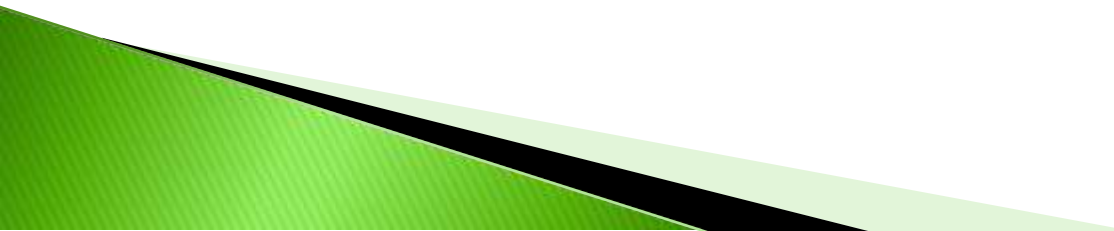
Generalization Relationship



Generalization Relationship



Aggregation Relationship

- ▶ Specialized form of association in which a whole is related to its part(s)
 - ▶ Represented by *a-part-of* relationship
- 

Aggregation Relationship

- ▶ Denoted by placing a diamond nearest the class representing the aggregation



Multiplicity

- Documents how many instances of a class can be associated with one instance of another class



Multiplicity

- Denotes the **minimum number.. maximum number** of instances

Exactly one 1

Zero or more 0..* or 0..m

One or more 1..* or 1..m

Zero or one 0..1

Specified range 2..4

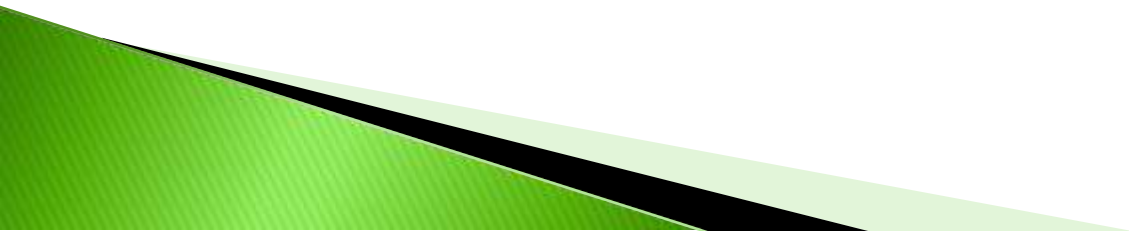
Multiple, disjoint ranges 1..3, 5

Class Diagram

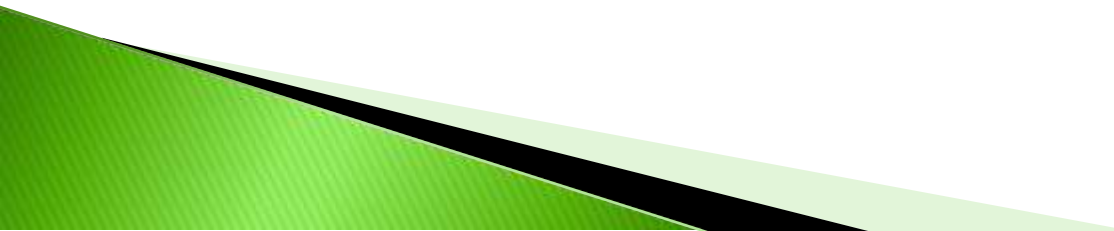
- ▶ 2 approaches to deriving class diagrams:
 - from use cases and their scenarios
 - from CRC cards

Deriving Class Diagram from Use Cases and Scenarios

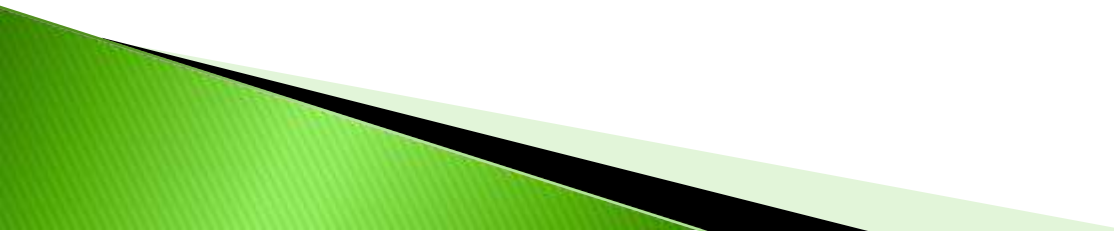
- ▶ Analyze the text in the use-case descriptions and scenarios



Guidelines for Analyzing Use Cases

- ▶ A common or improper noun implies a class of objects
 - ▶ A proper noun implies an instance of a class
 - ▶ A collective noun implies a class of objects made up of groups of instances of another class
- 

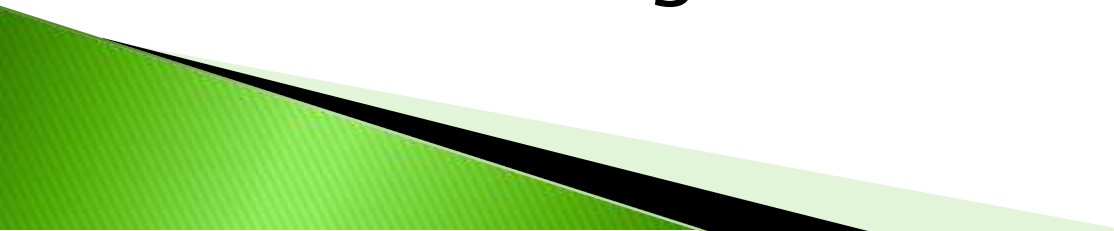
Guidelines for Analyzing Use Cases (2)

- ▶ An adjective implies an attribute of an object
 - ▶ A doing verb implies an operation
 - ▶ A being verb implies a relationship between an object and its class
 - ▶ A having verb implies an aggregation or association relationship
- 

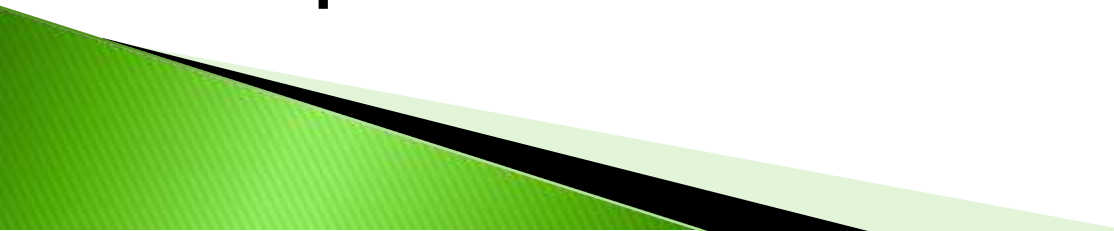
Guidelines for Analyzing Use Cases (3)

- ▶ A transitive verb implies an operation
 - ▶ An intransitive verb implies an exception
 - ▶ A predicate or descriptive verb phrase implies an operation
 - ▶ An adverb implies an attribute of a relationship or an operation
- 

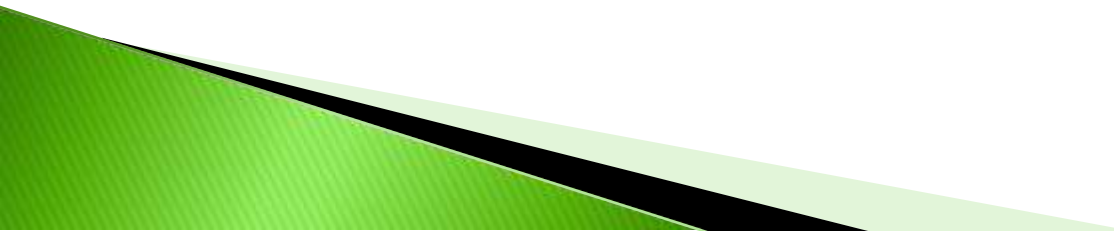
Class Diagram

- ▶ Ensure that the classes are both necessary and sufficient to solve the underlying problem
 - no missing attributes or methods in each class
 - no extra or unused attributes or methods in each class
 - no missing or extra classes
- 

Discarding Unnecessary and Incorrect Classes

- ▶ Redundant Classes
 - ▶ Irrelevant Classes
 - ▶ Vague Classes
 - ▶ Attributes
 - ▶ Operations
 - ▶ Roles
 - ▶ Implementation Constructs
- 

Types of Classes

- ▶ Ones found during analysis:
 - people, places, events, and things about which the system will capture information
 - ones found in application domain
 - ▶ Ones found during design
 - specific objects like windows and forms that are used to build the system
- 





END

